

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

ДЗ4: Технический дизайн микросервисной архитектуры

Выполнил:

Зеленков Евгений

Группа

БР2.2

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Разделить монолитное backend-приложение из ЛР1 на микросервисы с соблюдением концепции database-per-service.
2. Спроектировать микросервисную архитектуру, отразить взаимосвязи между сервисами и способы их взаимодействия.
3. Спроектировать разделение базы данных на обособленные БД отдельных сервисов.
4. Описать endpoints для межсервисного взаимодействия в формате OpenAPI.
5. Описать варианты запросов, ответов и возможные ошибки.
6. Сформировать документ с требованиями и шагами перехода от монолита к микросервисам.

Ход работы

Цель работы

Цель работы - спроектировать переход от монолитного REST API из ЛР1 к микросервисной архитектуре с соблюдением принципа database-per-service. В результате должен быть получен технический дизайн, который описывает границы сервисов, владение данными, взаимодействие между сервисами, внутренние API-контракты и шаги миграции.

Исходная система

В ЛР1 реализован монолитный backend сервиса бронирования столиков в ресторанах. Приложение построено на Express, TypeScript, TypeORM и PostgreSQL. В монолите реализованы следующие группы API:

- POST /api/v1/auth/register и POST /api/v1/auth/login - регистрация и авторизация;
- GET /api/v1/users/me - профиль текущего пользователя;
- /api/v1/restaurants - просмотр и администрирование ресторанов;
- /api/v1/restaurants/{restaurantId}/cuisines - кухни ресторана;
- /api/v1/restaurants/{restaurantId}/tables - столики ресторана;
- /api/v1/restaurants/{restaurantId}/menus и /api/v1/menus/{menuId}/items - меню и позиции меню;
- /api/v1/reservations - бронирования;
- /api/v1/reviews - отзывы и модерация отзывов.

В монолитной БД используются таблицы users, restaurants, restaurant_cuisines, restaurant_tables, menus, menu_items, reservations, reviews, photos_review, photos_menu_item. Внутри монолита эти таблицы связаны внешними ключами. При переходе к микросервисам прямые связи между БД разных сервисов должны быть заменены хранением внешних идентификаторов и проверками через API или события.

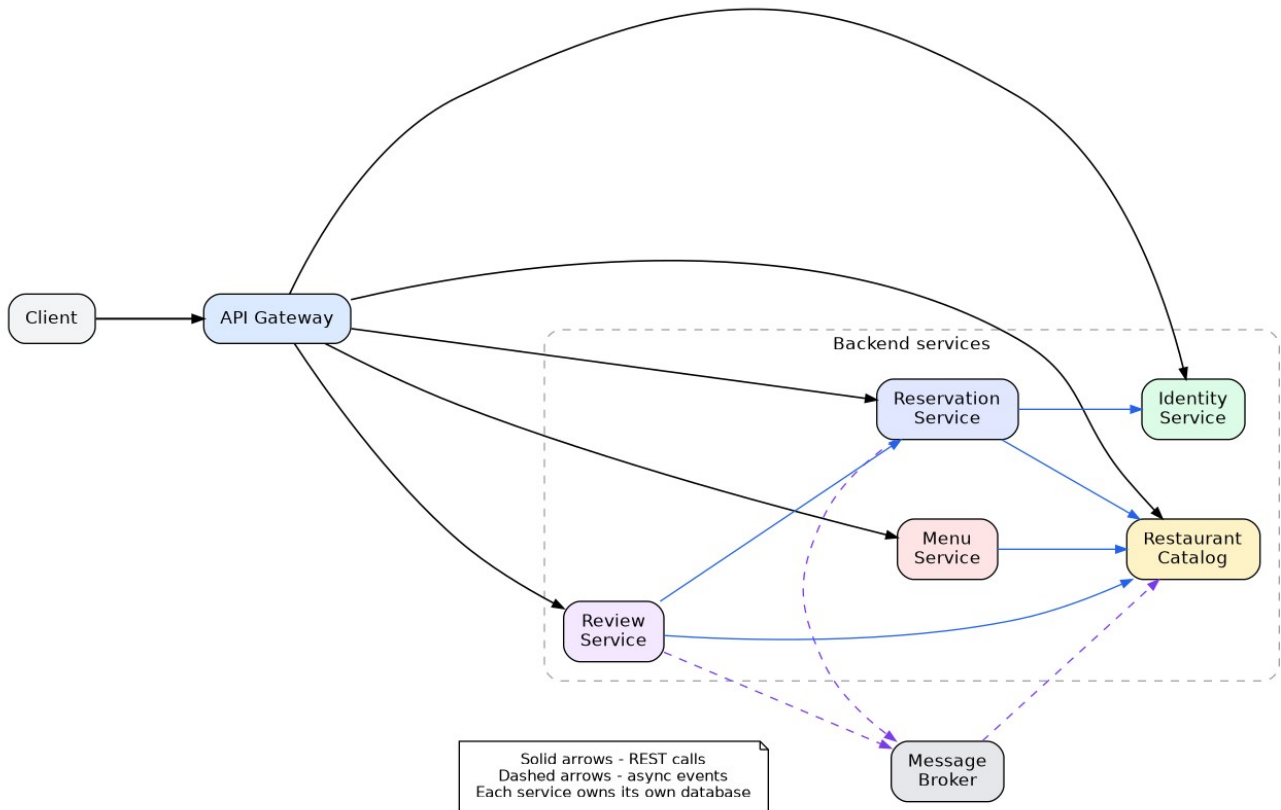
Целевая архитектура

Предлагается выделить шесть backend-компонентов:

1. api-gateway - единая внешняя точка входа для клиентского приложения.
2. identity-service - пользователи, роли, регистрация, вход, JWT.
3. restaurant-catalog-service - рестораны, кухни, столики.
4. menu-service - меню, блюда и фотографии блюд.
5. reservation-service - бронирования и жизненный цикл брони.
6. review-service - отзывы, фотографии отзывов, модерация.

Внешний клиент продолжает работать через /api/v1, но теперь запросы принимает api-gateway. Gateway проверяет JWT, прокидывает X-User-Id, X-User-Role и correlation id во внутренние сервисы, маршрутизирует запросы и скрывает от клиента внутреннюю топологию.

Общая схема взаимодействия:



Черные стрелки – синхронные REST-запросы. Синие сплошные стрелки – внутренние REST-запросы между сервисами. Фиолетовые пунктирные стрелки – асинхронные события через брокер сообщений.

Для синхронного взаимодействия используется REST поверх HTTP. Для событий, которые не должны блокировать пользовательский запрос, используется брокер сообщений. В рамках

проекта достаточно описать топики и payload событий, реализация может быть выполнена через RabbitMQ или Kafka.

Разделение по сервисам

API Gateway

Gateway не владеет доменными данными и не имеет собственной бизнесовой БД. Его ответственность:

- принимать внешние запросы /api/v1;
- проверять подпись JWT по публичному ключу или shared secret;
- маршрутизировать запросы во внутренние сервисы;
- нормализовать ошибки;
- прокидывать X-Request-Id, X-User-Id, X-User-Role;
- выполнять только заранее описанную легкую агрегацию read-only ответов без бизнес-правил.

Gateway не должен обращаться напрямую к БД сервисов. Это сохраняет изоляцию данных.

Gateway не должен превращаться в новый монолит. Бизнес-валидации остаются внутри доменных сервисов: бронирование проверяется в reservation-service, право на отзыв - в review-service, управление столиками - в restaurant-catalog-service. Агрегация в Gateway допустима только для чтения, например для публичной карточки ресторана, если клиенту нужен один ответ вместо трех запросов. В таком случае Gateway вызывает:

- restaurant-catalog-service для основной карточки ресторана;
- menu-service для опубликованных меню;
- review-service для последних подтвержденных отзывов.

Если menu-service или review-service недоступен, Gateway возвращает основную карточку ресторана и пустой блок menus или reviews с полем partial: true, потому что эти данные не критичны для просмотра ресторана. Если недоступен restaurant-catalog-service, Gateway возвращает 503 dependency_unavailable, так как без каталога нельзя собрать базовый ресурс. Для всех downstream-вызовов задается timeout 1-2 секунды, retry не более одного раза только для идиempотентных GET, а результат публичных GET /restaurants/{id} может кэшироваться на 30-60 секунд.

Identity Service

Сервис отвечает за учетные записи пользователей и роли.

Владеет БД identity_db:

- users: id, name, birthdate, phone, password_hash, is_verified, role, created_at, updated_at.

Основные операции:

- регистрация пользователя;
- вход по телефону и паролю;
- выдача JWT;
- получение и обновление профиля;
- внутренняя проверка существования пользователя и роли.

Другие сервисы не хранят ФИО, телефон и хеш пароля. В бронированиях и отзывах хранится только `user_id` как внешний идентификатор владельца.

В рамках лабораторной работы роль Admin считается глобальной ролью администратора платформы. Такой пользователь может управлять всеми ресторанами, меню, статусами бронирований и модерацией отзывов. Разграничение администраторов по конкретным ресторанам не реализуется. Если бы требовалась модель "администратор только своего ресторана", в `restaurant-catalog-service` понадобилась бы отдельная таблица `restaurant_admins(id, restaurant_id, user_id, role, created_at)`, а административные endpoints должны были бы проверять не только `Role.Admin`, но и связь пользователя с конкретным рестораном.

Restaurant Catalog Service

Сервис отвечает за справочник ресторанов, кухни и столики.

Владеет БД `restaurant_catalog_db`:

- `restaurants`: `id`, `name`, `foundation_date`, `phone`, `address`, `rating`, `avg_bill`, `is_verified`, `created_at`, `updated_at`;
- `restaurant_cuisines`: `id`, `restaurant_id`, `cuisine`, `created_at`, `updated_at`;
- `restaurant_tables`: `id`, `restaurant_id`, `number`, `capacity`, `is_active`, `created_at`, `updated_at`.

Основные операции:

- поиск и фильтрация ресторанов;
- получение карточки ресторана;
- управление ресторанами администратором;
- управление кухнями и столиками;
- внутренняя проверка существования ресторана;
- внутренняя проверка, что столик активен и принадлежит выбранному ресторану.

Сервис не знает о пользователях и бронированиях. Для рейтинга ресторана он принимает событие `review.verified` и пересчитывает агрегированное поле `rating`.

Menu Service

Сервис отвечает за меню ресторанов и позиции меню.

Владеет БД `menu_db`:

- `menus`: `id`, `restaurant_id`, `name`, `description`, `is_published`, `created_at`, `updated_at`;
- `menu_items`: `id`, `menu_id`, `name`, `price`, `description`, `is_published`, `created_at`, `updated_at`;

- photos_menu_item: id, menu_item_id, file_path, created_at, updated_at.

restaurant_id хранится как внешний идентификатор ресторана из restaurant-catalog-service, но FK между БД нет. При создании меню сервис синхронно проверяет ресторан через внутренний endpoint каталога.

Reservation Service

Сервис отвечает за создание, просмотр и изменение статусов бронирований.

Владеет БД reservation_db:

- reservations: id, user_id, restaurant_id, table_id, reserved_at, reserved_until, status, comment, guest_count, created_at, updated_at.

user_id, restaurant_id и table_id являются внешними идентификаторами. Сервис хранит их для быстрых выборок и аудита, но не делает SQL JOIN к чужим базам.

В монолите было поле reserved_at, но для корректного поиска пересечений одной точки времени недостаточно. В микросервисной версии добавляется reserved_until. Если клиент не передает длительность, сервис применяет бизнес-правило по умолчанию: бронь длится 2 часа. Тогда временной интервал брони задается диапазоном [reserved_at, reserved_until).

При создании брони сервис выполняет проверки:

1. пользователь существует и активен через identity-service;
2. ресторан существует через restaurant-catalog-service;
3. столик существует, активен, принадлежит ресторану и вмещает гостей через restaurant-catalog-service;
4. на выбранный столик нет пересекающейся активной брони в reservation_db.

Проверка конфликта и вставка брони должны выполняться атомарно внутри транзакции reservation-service. Для PostgreSQL основной вариант - exclusion constraint по table_id и диапазону времени:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

ALTER TABLE reservations
ADD CONSTRAINT reservations_no_active_overlap
EXCLUDE USING gist (
    table_id WITH =,
    tsrange(reserved_at, reserved_until, '[]') WITH &&
)
WHERE (status IN ('Pending', 'Confirmed'));
```

Так БД запрещает две активные брони на один столик с пересекающимися интервалами даже при одновременных запросах. Pending и Confirmed считаются активными статусами, Cancelled и Completed не блокируют новые брони. Если в будущем появится статус Seated, его также нужно добавить в активные. При нарушении constraint сервис возвращает 409 reservation_time_conflict.

Если вместо произвольных интервалов используются фиксированные слоты, допустим запасной вариант: хранить `slot_start`, `slot_duration_minutes` и завести частичный уникальный индекс по (`table_id`, `slot_start`) для активных статусов. Но для текущей модели с диапазоном времени `exclusion constraint` надежнее и точнее.

Жизненный цикл бронирования формализуется как `state machine`:

Текущий статус	Новый статус	Кто может выполнить	Комментарий
Pending	Confirmed	Admin	администратор подтверждает бронь
Pending	Cancelled	владелец брони или Admin	пользователь отменяет свою бронь или администратор отменяет бронь
Confirmed	Completed	Admin	администратор завершает бронь после посещения
Confirmed	Cancelled	владелец брони или Admin	отмена подтвержденной брони
Cancelled	terminal	никто	отмененную бронь нельзя вернуть в активный статус
Completed	terminal	никто	завершенную бронь нельзя изменить

Переход Pending -> Completed запрещен: перед завершением бронь должна быть подтверждена. Переходы из Cancelled и Completed запрещены, чтобы не ломать историю отзывать и расчет занятости столиков.

После изменения статуса сервис публикует событие `reservation.status_changed`, чтобы другие сервисы могли реагировать без синхронной связности.

Review Service

Сервис отвечает за отзывы и фотографии отзывов.

Владеет БД `review_db`:

- `reviews`: `id`, `user_id`, `restaurant_id`, `reservation_id`, `date`, `comment`, `rating`, `is_verified`, `created_at`, `updated_at`;
- `photos_review`: `id`, `review_id`, `file_path`, `created_at`, `updated_at`.

Перед созданием отзыва сервис проверяет бронирование через `reservation-service`. Отзыв можно создать только если:

- бронь существует;
- бронь принадлежит текущему пользователю;
- статус брони `Completed`;
- по этой брони еще нет отзыва.

После модерации отзыв публикует событие `review.verified`. `restaurant-catalog-service` использует это событие для пересчета рейтинга ресторана.

Разделение базы данных

В микросервисной архитектуре каждый сервис владеет своей БД и сам выполняет миграции. Прямой доступ к чужой БД запрещен.

Сервис	База данных	Таблицы	Внешние идентификаторы
identity-service	identity_db	users	нет
restaurant-catalog-service	restaurant_catalog_db	restaurants, restaurant_cuisines, restaurant_tables	нет
menu-service	menu_db	menus, menu_items, photos_menu_item	restaurant_id
reservation-service	reservation_db	reservations	user_id, restaurant_id, table_id
review-service	review_db	reviews, photos_review	user_id, restaurant_id, reservation_id

Такое разделение выбрано потому, что каждая группа таблиц имеет отдельный жизненный цикл. Например, бронирование часто меняет статус и требует проверки свободного времени столика, а меню изменяется администраторами независимо от бронирований. Если оставить меню и брони в одной БД, сервисы будут связаны общей схемой и реального database-per-service не получится.

Справочные enum-значения сохраняются в целевой архитектуре:

Enum	Значения	Где используется
Role	Admin, User	identity-service.users.role, JWT claims, Gateway authorization
ReservationStatus	Pending, Confirmed, Cancelled, Completed	reservation-service.reservations.status
Cuisine	European, Asian, Eastern, Caucasian, American, Vegetarian	restaurant-catalog-service.restaurant_cuisines.cuisine

Для ограничения пересечений броней важно не просто наличие enum, а явная классификация статусов. Активные статусы для блокировки времени: Pending, Confirmed. Неактивные статусы: Cancelled, Completed.

Маршрутизация внешнего API

Старый внешний контракт /api/v1 сохраняется. Клиент не должен знать, что монолит разделен на сервисы. Gateway маршрутизирует запросы так:

Внешний endpoint	Целевой сервис	Комментарий
POST /api/v1/auth/register	identity-service	регистрация пользователя
POST /api/v1/auth/login	identity-service	выдача JWT
GET /api/v1/users/me	identity-service	профиль текущего пользователя
GET /api/v1/restaurants	restaurant-catalog-service	список и фильтрация ресторанов
POST /api/v1/restaurants	restaurant-catalog-service	административное создание ресторана
GET /api/v1/restaurants/{id}	restaurant-catalog-service или Gateway aggregation	базовая карточка из catalog, опционально меню и отзывы отдельными блоками
GET /api/v1/restaurants/{id}/cuisines	restaurant-catalog-service	кухни ресторана
GET /api/v1/restaurants/{id}/tables	restaurant-catalog-service	активные столики ресторана
GET /api/v1/restaurants/{id}/menus	menu-service	меню фильтруется по restaurant_id
GET /api/v1/menus/{menuId}/items	menu-service	позиции меню
POST /api/v1/reservations	reservation-service	создание брони
GET /api/v1/reservations	reservation-service	список броней пользователя или администратора
PATCH /api/v1/reservations/{id}/cancel	reservation-service	отмена брони
PATCH /api/v1/reservations/{id}/status	reservation-service	административная смена статуса

GET /api/v1/restaurants/{id}/reviews	review-service	публичные проверенные отзывы
POST /api/v1/reviews	review-service	создание отзыва
PATCH /api/v1/reviews/{id}/verification	review-service	модерация отзыва

Межсервисное взаимодействие

Синхронные REST-запросы

Синхронные вызовы используются там, где результат нужен для немедленного ответа пользователю:

- reservation-service -> identity-service: проверка пользователя при создании брони;
- reservation-service -> restaurant-catalog-service: проверка ресторана и столика;
- menu-service -> restaurant-catalog-service: проверка ресторана перед созданием меню;
- review-service -> reservation-service: проверка завершенной брони перед созданием отзыва;
- review-service -> restaurant-catalog-service: проверка ресторана при выдаче списка отзывов.

Для внутренних вызовов используется префикс /internal/v1. Эти endpoints не публикуются наружу через Gateway.

Синхронные зависимости являются критичными только для операций записи. Если identity-service недоступен, создание брони и отзыва должно возвращать вызывающему клиенту 503 `dependency_unavailable`, потому что сервис не может надежно проверить пользователя. Если restaurant-catalog-service недоступен, создание брони и меню также возвращает 503. Если зависимый сервис не ответил за `timeout`, вызывающий сервис или Gateway возвращает 504 `dependency_timeout`. Для публичных `read-only` запросов допустим короткий кэш: данные пользователя не кэшируются для авторизации, но результат проверки существования пользователя можно кэшировать на 30 секунд; данные ресторана и столика можно кэшировать на 30-60 секунд только при подписке на события изменения столиков и ресторанов.

Асинхронные события

Асинхронные события используются для обновлений, которые не должны блокировать основной запрос:

- reservation.status_changed - публикуется при смене статуса бронирования;
- review.verified - публикуется после подтверждения отзыва администратором;
- restaurant.rating_recalculated - может публиковаться после обновления рейтинга ресторана.

Payload события review.verified:

```
{
  "eventId": "uuid",
  "occurredAt": "2026-03-20T12:00:00Z",
  "reviewId": "uuid",
  "restaurantId": "uuid",
  "reservationId": "uuid",
  "rating": 4.5
}
```

События должны быть идемпотентными: потребитель хранит eventId обработанных событий или использует уникальные ключи, чтобы повторная доставка не приводила к двойному пересчету.

Поле restaurants.rating в restaurant-catalog-service является денормализованной read model, а источником истины по отзывам остается review-service. Для восстановления после сбоя возможен replay событий или периодическая сверка с review-service, который может отдавать агрегированные значения rating_avg и rating_count по restaurant_id. Если отзыв изменяется, удаляется или снимается с публикации, должны публиковаться дополнительные события review.updated, review.deleted или review.unverified.

Внутренние OpenAPI-контракты

Для межсервисного взаимодействия подготовлена спецификация openapi-internal.yaml. В ней описаны ключевые внутренние endpoints:

- GET /internal/v1/users/{userId} - получить минимальную информацию о пользователе;
- POST /internal/v1/users/validate - проверить существование пользователя и роль;
- GET /internal/v1/restaurants/{restaurantId} - получить ресторан;
- POST /internal/v1/restaurants/{restaurantId}/tables/validate - проверить столик ресторана;
- GET /internal/v1/reservations/{reservationId} - получить бронь;
- POST /internal/v1/reservations/{reservationId}/validate-review-eligibility - проверить право на отзыв;
- POST /internal/v1/restaurants/{restaurantId}/rating/recalculate - пересчитать рейтинг после модерации отзыва.

Внутренние ответы возвращают только данные, необходимые вызывающему сервису.

Например, при проверке пользователя не возвращается телефон и хеш пароля, а при проверке столика возвращаются tableId, restaurantId, capacity и isActive.

Фрагмент OpenAPI для проверки столика:

```
paths:
  /internal/v1/restaurants/{restaurantId}/tables/validate:
    post:
      summary: Validate table ownership, activity and capacity
      parameters:
        - name: restaurantId
          in: path
          required: true
          schema:
            type: string
            format: uuid
        - name: X-Request-Id
          in: header
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
```

```

    application/json:
      schema:
        type: object
        required: [tableId, guestCount]
        properties:
          tableId:
            type: string
            format: uuid
          guestCount:
            type: integer
            minimum: 1
        example:
          tableId: c8c6b9be-f02a-4dd6-9f6a-50efaef2a4cf
          guestCount: 4
  responses:
    '200':
      description: Table can be used for reservation
      content:
        application/json:
          example:
            valid: true
            table:
              id: c8c6b9be-f02a-4dd6-9f6a-50efaef2a4cf
              restaurantId: 6ef2e9a4-3c0b-4fd4-8e27-61ffb0fd96d4
              capacity: 4
              isActive: true
    '409':
      description: Table cannot be used
      content:
        application/json:
          examples:
            mismatch:
              value:
                code: table_restaurant_mismatch
                message: Table does not belong to restaurant
            inactive:
              value:
                code: table_inactive
                message: Table is inactive
            capacity:
              value:
                code: table_capacity_too_small
                message: Table capacity is too small

```

Если reservation-service вызывает этот endpoint и restaurant-catalog-service недоступен, то 503 dependency_unavailable формирует вызывающий сервис или Gateway. Сам endpoint restaurant-catalog-service описывает ответы 200, 404, 409, 422, 500.

Фрагмент OpenAPI для проверки права на отзыв:

```

paths:
  /internal/v1/reservations/{reservationId}/validate-review-eligibility:
    post:
      summary: Validate that a reservation can be used to create a review
      parameters:
        - name: reservationId
          in: path
          required: true
          schema:
            type: string
            format: uuid

```

```

- name: X-Request-Id
  in: header
  required: true
  schema:
    type: string
    format: uuid
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required: [userId]
        properties:
          userId:
            type: string
            format: uuid
      example:
        userId: 6d8f8e3a-f7e6-4ad1-89d3-c4c0bbd4575f
responses:
  '200':
    description: Review can be created
    content:
      application/json:
        example:
          eligible: true
          reservation:
            id: 3f7ac73a-54d0-46e3-bc94-861390b16b47
            userId: 6d8f8e3a-f7e6-4ad1-89d3-c4c0bbd4575f
            restaurantId: 6ef2e9a4-3c0b-4fd4-8e27-61ffb0fd96d4
            tableId: c8c6b9be-f02a-4dd6-9f6a-50efaef2a4cf
            reservedAt: '2026-03-20T19:00:00Z'
            reservedUntil: '2026-03-20T21:00:00Z'
            status: Completed
            guestCount: 4
  '403':
    description: Reservation belongs to another user
    content:
      application/json:
        example:
          code: forbidden
          message: Reservation belongs to another user
  '409':
    description: Reservation is not completed or review already exists
    content:
      application/json:
        examples:
          notCompleted:
            value:
              code: reservation_not_completed
              message: Reservation must be completed before review
          duplicate:
            value:
              code: review_already_exists
              message: Review for reservation already exists

```

Если review-service вызывает этот endpoint и reservation-service не отвечает за timeout, то 504 dependency_timeout формирует вызывающий сервис или Gateway. Сам endpoint reservation-service в своей OpenAPI-схеме описывает только ответы, которые он реально может вернуть после обработки запроса: 200, 403, 404, 409, 422, 500.

Пример сценария создания брони

1. Клиент отправляет POST /api/v1/reservations в api-gateway.
2. Gateway проверяет JWT и передает запрос в reservation-service с заголовками X-User-Id и X-User-Role.
3. reservation-service вызывает POST /internal/v1/users/validate в identity-service.
4. reservation-service вызывает GET /internal/v1/restaurants/{restaurantId} в restaurant-catalog-service.
5. reservation-service вызывает POST /internal/v1/restaurants/{restaurantId}/tables/validate в restaurant-catalog-service.
6. reservation-service открывает транзакцию и пытается вставить бронь с диапазоном [reserved_at, reserved_until).
7. PostgreSQL exclusion constraint атомарно запрещает пересечение с активными статусами Pending и Confirmed.
8. Клиент получает 201 Created и данные бронирования.

Если ресторан или столик не найдены, возвращается 404. Если столик не принадлежит ресторану, неактивен, не вмещает гостей или уже занят на пересекающийся интервал, возвращается 409. Если зависимый сервис недоступен, возвращается 503; если он не ответил за timeout, возвращается 504. Если тело запроса некорректно, возвращается 422.

Пример сценария создания отзыва

1. Клиент отправляет POST /api/v1/reviews.
2. Gateway проверяет JWT и передает запрос в review-service.
3. review-service вызывает POST /internal/v1/reservations/{reservationId}/validate-review-eligibility.
4. reservation-service проверяет владельца брони и статус Completed.
5. review-service проверяет, что по reservation_id еще нет отзыва.
6. review-service сохраняет отзыв со значением is_verified = false.
7. После модерации администратор вызывает PATCH /api/v1/reviews/{reviewId}/verification.
8. review-service публикует событие review.verified.
9. restaurant-catalog-service получает событие и пересчитывает restaurants.rating.

Так рейтинг ресторана обновляется асинхронно. Это лучше, чем синхронно пересчитывать рейтинг во время модерации, потому что сбой каталога ресторанов не должен отменять успешное подтверждение отзыва.

Ошибки

Все сервисы используют единый формат ошибки:

```
{
  "code": "validation_error",
  "message": "Request validation failed",
  "fields": [
    {
      "name": "guestCount",
      "message": "guestCount must be greater than zero"
    }
  ]
}
```

}

Типовые ошибки:

HTTP статус	Код	Когда возникает
400	bad_request	некорректный JSON или неверный формат параметра
401	unauthorized	нет JWT или внутреннего service token
403	forbidden	роль пользователя не позволяет выполнить действие
404	user_not_found	пользователь не найден
404	restaurant_not_found	ресторан не найден
404	table_not_found	стол не найден
404	reservation_not_found	бронь не найдена
409	table_restaurant_mismatch	стол не принадлежит ресторану
409	table_inactive	стол выключен администратором
409	table_capacity_too_small	вместимость стола меньше числа гостей
409	reservation_time_conflict	на это время уже есть активная бронь
409	reservation_not_completed	отзыв создается по незавершенной брони
409	review_already_exists	по брони уже создан отзыв
422	validation_error	ошибка валидации полей
424	failed_dependency	используется только в составных операциях, если предыдущий обязательный шаг уже завершился ошибкой
503	dependency_unavailable	необходимый сервис временно недоступен
504	dependency_timeout	необходимый сервис не ответил за заданный timeout
500	internal_error	непредвиденная ошибка сервиса

Доменные ограничения не кодируются через 424. Если стол неактивен, не принадлежит ресторану, не хватает вместимости, бронь не завершена или отзыв уже существует, сервис возвращает 409 Conflict с предметным кодом ошибки.

Требования к безопасности и надежности

- Внешние запросы проходят только через api-gateway.
- Внутренние endpoints защищаются service token или mTLS.
- JWT не передается как источник истины между сервисами без проверки Gateway; сервисы получают нормализованные X-User-Id и X-User-Role.
- Внутренние сервисы доверяют заголовкам X-User-Id и X-User-Role только если запрос пришел от Gateway или другого доверенного сервиса и прошел проверку service token/mTLS.
- Внутренние endpoints недоступны из внешней сети на уровне network policy, firewall или private subnet, поэтому внешний клиент не может напрямую подделать X-User-Role: Admin.
- Все запросы содержат X-Request-Id для трассировки.
- Для межсервисных вызовов задаются timeout, retry с ограничением и circuit breaker.
- События публикуются через outbox-паттерн: сервис сначала сохраняет изменение и событие в своей БД в одной транзакции, затем отдельный publisher отправляет событие в брокер.
- Схемы событий версионизируются через поле eventVersion.

План миграции от монолита

Миграция выполняется по подходу *strangler fig pattern*: новый контур постепенно "обрастает" вокруг монолита, а Gateway по одному маршруту переключает трафик на новые сервисы.

1. Зафиксировать текущий внешний API и поставить Gateway перед монолитом. На первом этапе Gateway просто проксирует существующие маршруты `/api/v1` в монолит.
2. Выделить первый сервис, например `identity-service`, и запустить его параллельно с монолитом в `shadow/read-only` режиме.
3. Перенести данные из монолитной БД в БД нового сервиса миграционными скриптами.
4. Сверить результат миграции: количество записей, контрольные суммы по ключевым полям, выборочные интеграционные проверки API.
5. Переключить чтение соответствующих маршрутов Gateway на новый сервис.
6. После проверки чтения переключить запись. Для операций записи на время миграции нужен один источник истины, чтобы не получить расхождение данных.
7. Повторить процесс для `restaurant-catalog-service`, `menu-service`, `reservation-service` и `review-service`.
8. Добавить брокер сообщений и `outbox` для событий `review.verified` и `reservation.status_changed`.
9. Настроить мониторинг: `health checks`, логи с `correlation id`, метрики `latency/error rate` по каждому сервису.
10. Если после переключения обнаруживаются ошибки, Gateway временно возвращает маршрут на монолит, данные нового сервиса повторно синхронизируются, после чего переключение повторяется.
11. После успешной проверки всех маршрутов отключить прямые монолитные `routes` и оставить обслуживание через микросервисы.

Почему выбрано такое разбиение

Сервисы выделены не по техническим слоям, а по бизнес-возможностям и владению данными. Это позволяет каждому сервису иметь свою БД, свои миграции и свой жизненный цикл разработки.

Не выделялся отдельный `table-service`, потому что столики являются частью каталога ресторана: они создаются и изменяются вместе с рестораном, а не живут как самостоятельный бизнес-домен.

Не выделялся отдельный `photo-service`, потому что в текущей системе фотографии представлены только ссылками `file_path`. Хранение файлов можно вынести во внешнее `object storage`, а метаданные фотографий оставить рядом с владельцем: фото блюда в `menu-service`, фото отзыва в `review-service`.

Не выбран вариант с общей БД и несколькими сервисами поверх нее, потому что это нарушило бы `database-per-service`. В таком варианте сервисы остались бы связаны общей схемой, FK и миграциями, а изменение одной области могло бы ломать другие.

Не выбран полностью событийный вариант без синхронных REST-проверок, потому что создание брони и отзыва требует немедленной валидации и понятного ответа клиенту. События используются там, где допустима eventual consistency: обновление рейтинга и реакция на смену статуса брони.

Вывод

В ходе работы был спроектирован переход монолитного backend-приложения бронирования ресторанов к микросервисной архитектуре. Были выделены сервисы по бизнес-границам, описано разделение БД по принципу database-per-service, определены синхронные REST-вызовы и асинхронные события, подготовлена внутренняя OpenAPI-спецификация и описан пошаговый план миграции. Предложенная архитектура сохраняет внешний API для клиента, но устраняет общую БД и прямые зависимости между доменными областями.

Приложение: OpenAPI

Полная внутренняя OpenAPI-спецификация сохранена в файле hw4/openapi-internal.yaml. В отчете дополнительно приведены два ключевых фрагмента OpenAPI с request, response и error examples.